МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МОЭВМ

ОТЧЕТ

по практической работе № 2

по дисциплине «Системы параллельной обработки данных»

Тема: «Пляжный волейбол»

Студент гр. 2304 _____ Пархоменко М. С.
подпись

Преподаватель _____ Татаринов Ю. С.
подпись

Санкт-Петербург
2026

1 Цель работы

Изучить обмен сообщениями средствами MPI, реализовать случайный выбор процесса-адресата и исследовать работу параллельной программы при различном числе процессов и различном размере сообщения.

2 Формулировка задания

Вариант 13. Пляжный волейбол. Процесс 0 генерирует сообщение и посылает его любому другому процессу группы. Процесс-приёмник передаёт сообщение дальше. Выбор процесса-адресата осуществляется случайным образом.

3 Описание алгоритма

В программе участвуют p MPI-процессов с рангами от 0 до $p - 1$, причём для выполнения обмена необходимо $p \geq 2$. После инициализации MPI процесс 0 формирует исходное сообщение и случайным образом выбирает первый процесс-получатель среди процессов с рангами от 1 до $p - 1$. Остальные процессы ожидают поступления сообщения.

Получив сообщение, процесс определяет ранг следующего адресата. Адресат выбирается из всех процессов группы, кроме самого отправителя, поэтому сообщение не может быть немедленно послано процессом самому себе. Затем полученное сообщение передаётся выбранному процессу, который повторяет те же действия. Таким образом, в каждый момент обмена один процесс владеет сообщением, а последовательность его перемещений между процессами заранее не определена.

В качестве сообщения используется целое число, обозначающее номер передачи. Процесс 0 начинает обмен со значения 1. Каждый следующий получатель увеличивает число на единицу перед дальнейшей отправкой. В текущей версии игра ограничена десятью передачами.

Обычное сообщение передаётся с тегом TAG_BALL. После получения сообщения с номером 10 процесс не передаёт мяч дальше, а рассылает всем остальным процессам управляющее сообщение с тегом TAG_STOP. Получив этот тег, процесс выходит из цикла приёма. Такой протокол позволяет завершить не только последний процесс-получатель, но и процессы, ожидающие сообщение в MPI_Recv.

Случайный адресат выбирается повторной генерацией числа в диапазоне $[0, p - 1]$ до тех пор, пока полученный ранг не будет отличаться от ранга отправителя. Генератор псевдослучайных чисел каждого процесса получает начальное значение, зависящее от текущего времени и ранга процесса.

4 Формальное описание с помощью сетей Петри

Алгоритм формализован цветной сетью Петри с опорой на основные положения аппарата сетей Петри [1]. Пусть $R = \{0, \dots, p-1\}$ --- множество рангов, H --- заданное число передач, m --- содержимое сообщения. Место Z содержит начальную фишку процесса 0. Цветная фишка места A имеет вид (i, k, m) и означает, что процесс i получил сообщение после передачи с номером k. Место Q_r содержит уведомление об остановке процесса r, а F_r обозначает его завершение. Обобщённая схема сети показана на рисунке 1.

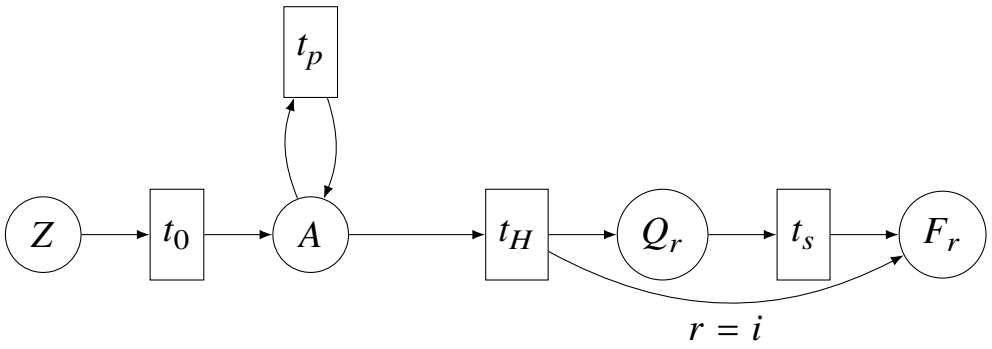


Рисунок 1 – Обобщённая сеть Петри алгоритма передачи сообщения

Функционалы переходов имеют вид

$$Z \xrightarrow{t_0} A(j, 1, m), \qquad j \in R \setminus \{0\}; \tag{1}$$

$$A(i, k, m) \xrightarrow{t_p} A(j, k+1, m), \qquad k < H, \quad j \in R \setminus \{i\}; \tag{2}$$

$$A(i, H, m) \xrightarrow{t_H} F_i + \sum_{r \in R \setminus \{i\}} Q_r; \tag{3}$$

$$Q_r \xrightarrow{t_s} F_r. \tag{4}$$

t_0 создаёт сообщение, t_p случайно выбирает адресата и передаёт ему сообщение, а t_H и t_s завершают процессы. Единственная фишка места A гарантирует наличие одного «мяча» и последовательность передач; условие $j \neq i$ исключает отправку самому себе. Итоговая маркировка содержит F_r для каждого r, поэтому после остановки заблокированных процессов нет.

5 Реализация и функциональная проверка

Программа реализована на языке C с использованием MPI. Полный листинг текущей версии приведён в приложении А. Для сборки в конфигурацию CMake добавлена цель volleyball. Используется стандарт C11, библиотека MPI::MPI_C и предупреждения компилятора -Wall, -Wextra, -Wpedantic. Сборка выполнена без предупреждений. Сборка цели и результат запуска на двух процессах представлены на рисунке 2.

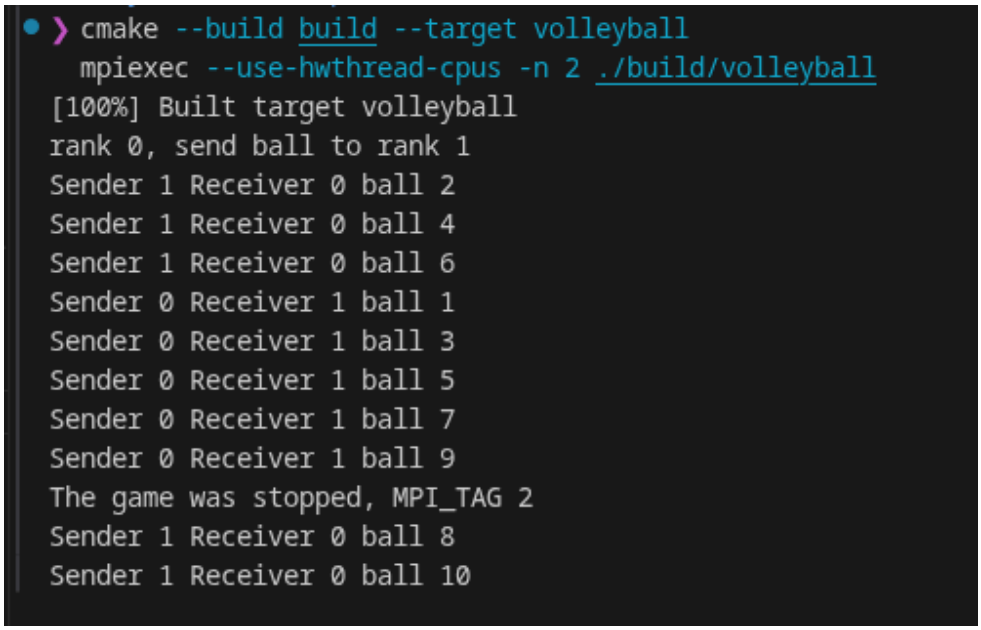


Рисунок 2 – Сборка программы и запуск на двух процессах

После начальной отправки все процессы, включая процесс 0, входят в общий цикл приёма. Вызов MPI_Recv использует MPI_ANY_SOURCE и MPI_ANY_TAG. Фактический отправитель и тег определяются по структуре MPI_Status. Различение обычных и управляющих сообщений выполняется по тегам TAG_BALL и TAG_STOP.

Для функциональной проверки выполнены запуски на двух, четырёх и восьми процессах. Во всех случаях программа выполнила передачи с номерами от 1 до 10, не выбирала текущий процесс в качестве адресата и завершилась с нулевым кодом возврата. Результаты приведены в таблице 1.

Таблица 1 – Результаты функциональной проверки

Число процессов	Число передач	Сообщений остановки	Результат
2	10	1	Успешно
4	10	3	Успешно
8	10	7	Успешно

Полный диагностический вывод запуска на восьми процессах показан на рисунке 3.

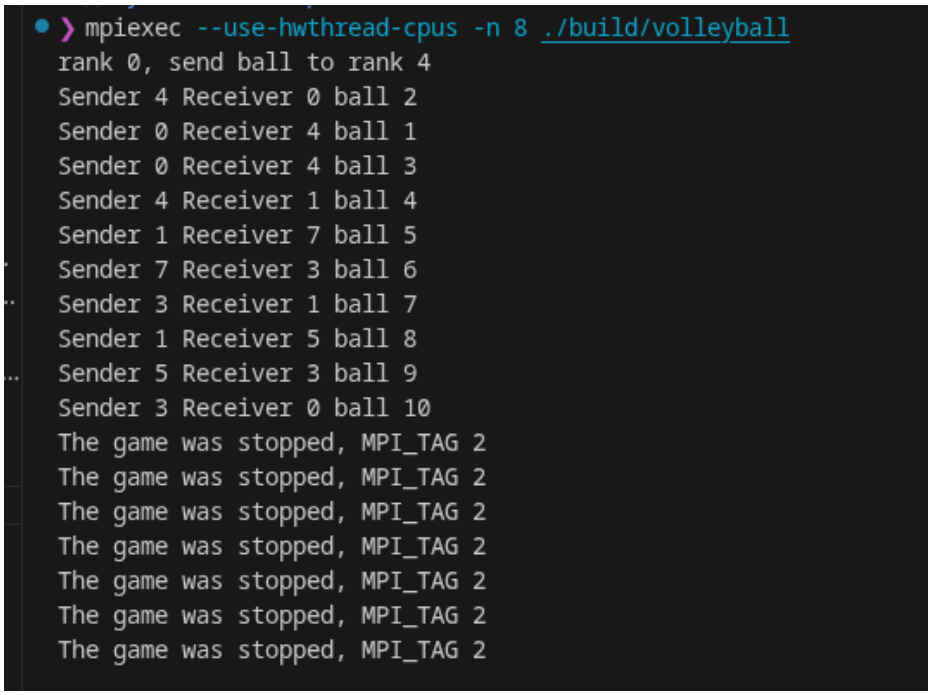


Рисунок 3 – Запуск программы на восьми процессах

В этом запуске получена следующая последовательность передачи мяча:

$$0 \to 4 \to 0 \to 4 \to 1 \to 7 \to 3 \to 1 \to 5 \to 3 \to 0.$$

Она содержит десять передач, причём соседние ранги различаются. При повторном запуске последовательность может измениться. Строки диагностического вывода разных процессов также могут отображаться не в порядке номеров передач, поскольку MPI не задаёт общий порядок вывода в стандартный поток.

6 Экспериментальное исследование

6.1. Методика измерения

Для измерений создана отдельная программа, листинг которой приведён в приложении Б. Сообщение состоит из одного целого числа со счётчиком передач и n целых чисел полезной нагрузки. Исследованы значения $n = 1$, 1024 и 65536, то есть 4 байта, 4 Кбайт и 256 Кбайт полезных данных. В каждом опыте выполнялось $H = 500$ передач.

Буферы заполнялись до начала измерения. Перед каждой серией процессы синхронизировались вызовом MPI_Barrier, время определялось функцией MPI_Wtime. После рассылки сообщений остановки выполнялась ещё одна барьерная синхронизация. Результатом опыта считался максимум локальных времён, полученный операцией MPI_Reduce с MPI_MAX. Формирование данных и печать результатов в измеряемый интервал не входили. Использованные операции соответствуют стандарту MPI [2].

Для каждой пары (p, n) выполнены два прогревочных и семь измерительных запусков внутри одного MPI-сеанса; итоговым временем считалась медиана. Для воспроизводимости маршрутов использовалось фиксированное начальное значение генератора, различающееся для разных рангов. Исследованы $p = 1, 2, 4, 8, 16$. При $p \geq 2$ выполнялся алгоритм передачи мяча. Так как исходная постановка не допускает обмен при $p = 1$, последовательная база моделировалась 500 локальными копированиями того же буфера. Поэтому ускорение относительно $p = 1$ характеризует стоимость межпроцессного обмена, но не является классическим ускорением одной и той же MPI-программы.

Измерения проводились на Intel Core i5-12400F: шесть физических ядер и 12 аппаратных потоков. Использовались Linux 7.3.1-1-cachyos, Open MPI 5.0.11 и GCC 16.2.1, конфигурация Release. Для $p = 16$ применялась переподписка, поэтому эта точка не характеризует выполнение на шестнадцати

независимых аппаратных потоках.

Ускорение и эффективность вычислялись по формулам

$$S(p,n) = \frac{T(1,n)}{T(p,n)}, \tag{5}$$

$$E(p,n) = \frac{S(p,n)}{p}. \tag{6}$$

6.2. Результаты измерений

Медианное время приведено в таблице 2. Исходные значения всех повторов и сводная таблица сохранены в CSV-файлах в каталоге pr2/results.

Таблица 2 – Медианное время 500 передач, мс

n	$p = 1$	$p = 2$	$p = 4$	$p = 8$	$p = 16$
1	0,012	0,249	0,569	0,390	0,798
1024	0,048	0,705	4,027	3,040	1,438
65536	2,316	8,935	8,920	9,241	17,563

Зависимость времени от числа процессов показана на рисунке 4. Точки $p = 1$ соответствуют локальной последовательной базе.

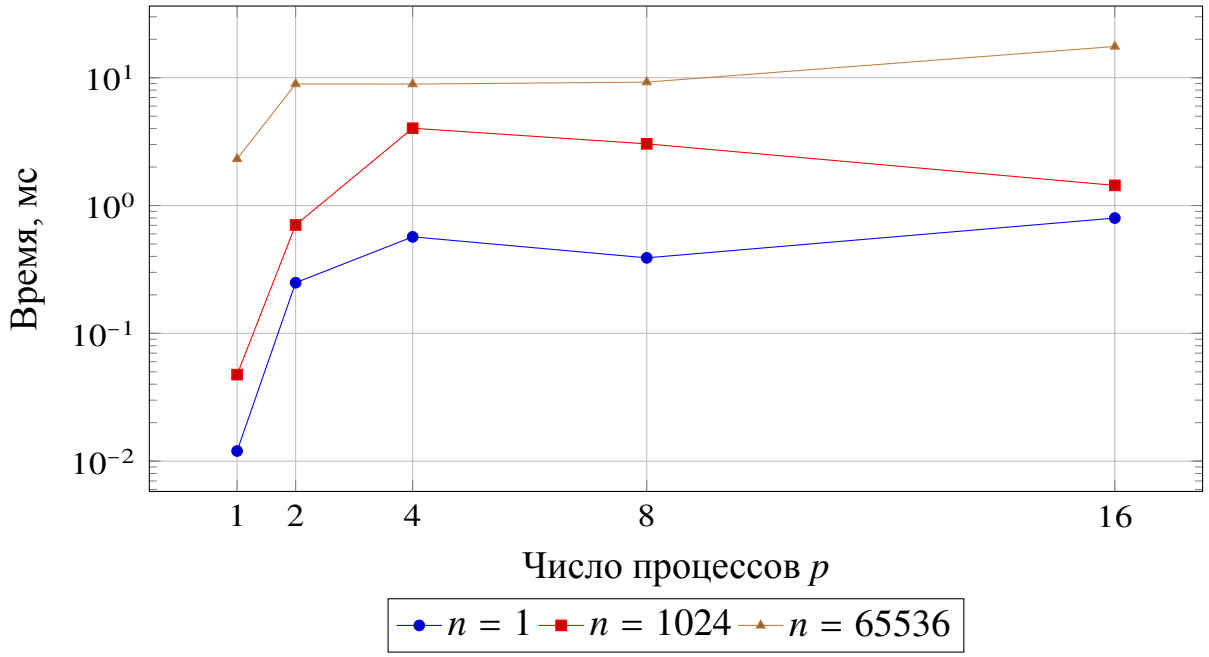


Рисунок 4 – Зависимость времени от числа процессов

Влияние размера полезной нагрузки показано на рисунке 5. Обе оси представлены в логарифмическом масштабе.

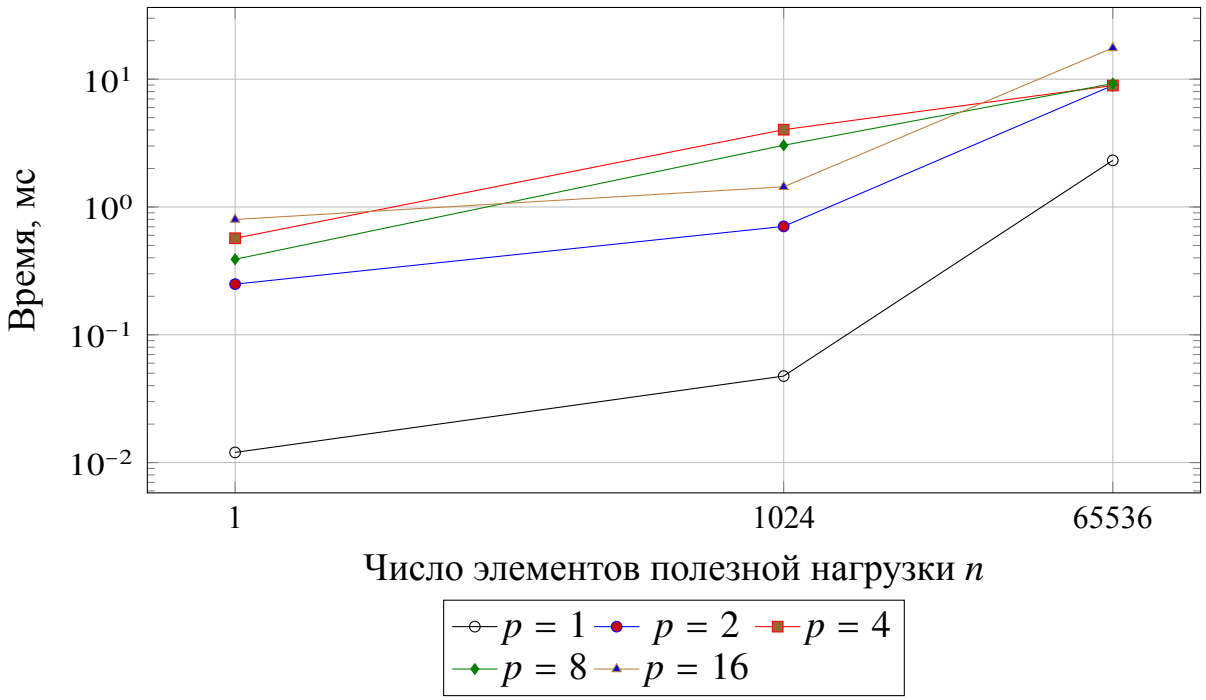


Рисунок 5 – Зависимость времени от размера полезной нагрузки

Значения ускорения по формуле (5) приведены в таблице 3 и на рисунке 6.

Таблица 3 – Ускорение относительно локальной последовательной базы

n	$p = 1$	$p = 2$	$p = 4$	$p = 8$	$p = 16$
1	1,000	0,048	0,021	0,031	0,015
1024	1,000	0,067	0,012	0,016	0,033
65536	1,000	0,259	0,260	0,251	0,132

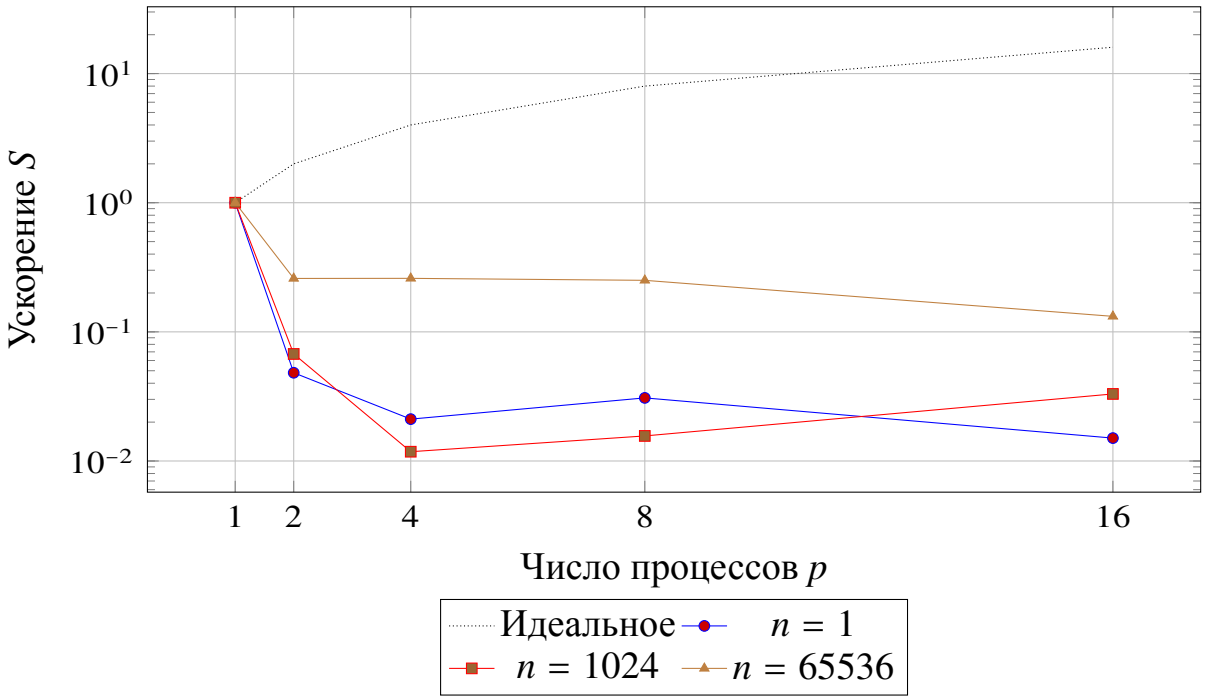


Рисунок 6 – Ускорение относительно локальной последовательной базы

Эффективность по формуле (6) приведена в таблице 4 и на рисунке 7.

Таблица 4 – Эффективность использования процессов

n	$p = 1$	$p = 2$	$p = 4$	$p = 8$	$p = 16$
1	1,000	0,024	0,005	0,004	0,001
1024	1,000	0,034	0,003	0,002	0,002
65536	1,000	0,130	0,065	0,031	0,008

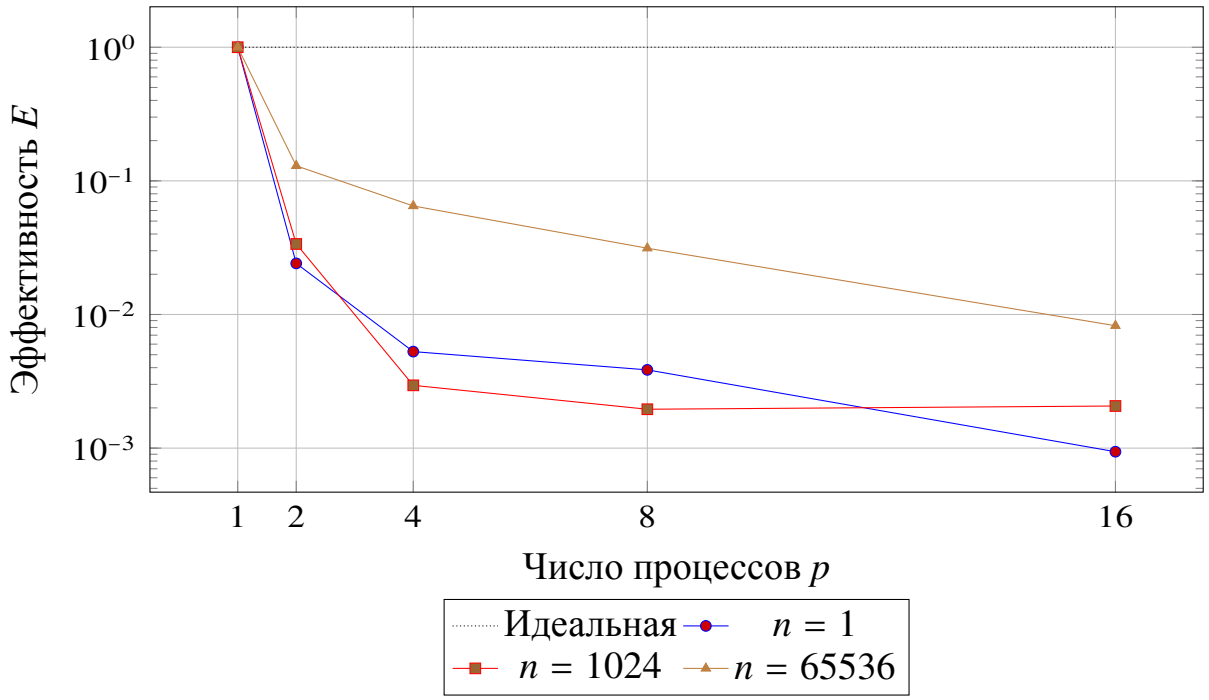


Рисунок 7 – Эффективность использования процессов

6.3. Анализ результатов

Во всех опытах с $p > 1$ ускорение оказалось меньше единицы. Причина следует из модели сети Петри: в месте A находится единственная фишка, поэтому в каждый момент времени сообщение обрабатывает только один процесс. Увеличение числа процессов расширяет множество возможных адресатов, но не создаёт параллельно выполняемых передач. По этой причине зона эффективного использования нескольких процессов для данной постановки не обнаружена.

Для $n = 65536$ время при $p = 2$ и $p = 4$ практически совпало: 8,935 и 8,920 мс, различие составляет менее 0,2%. При $p = 8$ время возросло до 9,241 мс, а при $p = 16$ --- до 17,563 мс. Последняя точка дополнительно искажена переподпиской. Максимальное наблюдаемое ускорение среди режимов MPI составило только 0,260 при $n = 65536$ и $p = 4$.

При малом размере сообщения основную долю времени определяют задержка MPI, синхронизация и планирование, поэтому зависимость от p немонотонна. Например, при $n = 1024$ медиана для $p = 4$ равна 4,027 мс,

а для $p = 16$ --- 1,438 мс. Это не является параллельным ускорением: обе точки существенно медленнее локальной базы 0,048 мс. Для $n = 1$, $p = 16$ также наблюдался единичный выброс до 7,970 мс при медиане 0,798 мс, что подтверждает влияние планирования в режиме переподписки.

Рост n увеличивает время передачи во всех режимах. При $p = 2$ увеличение полезной нагрузки с 4 байт до 256 Кбайт повысило медианное время с 0,249 до 8,935 мс. Для больших сообщений влияние собственно копирования данных становится заметнее, поэтому отношение времени MPI к локальной базе меньше, чем для коротких сообщений. Однако последовательный характер движения одного мяча не позволяет превратить это уменьшение относительных накладных расходов в ускорение.

7 Выводы

В работе реализован алгоритм «Пляжный волейбол»: процесс 0 создаёт сообщение, а каждый получатель случайно выбирает следующего адресата, отличного от себя. Счётчик передач и два тега сообщений обеспечивают ограничение длительности игры и корректное завершение всех MPI-процессов. Функциональные запуски на двух, четырёх и восьми процессах подтвердили выполнение десяти передач без зависаний и дублирования мяча.

Алгоритм формализован цветной сетью Петри. Единственная фишка активного сообщения показывает, что передачи принципиально последовательны. Этот вывод подтверждён измерениями 500 передач для трёх размеров полезной нагрузки и $p = 1, 2, 4, 8, 16$.

Ни в одной MPI-конфигурации не получено ускорение относительно локального копирования. Наибольшее отношение $T(1, n)/T(p, n)$ равно 0,260 для сообщения размером 256 Кбайт при четырёх процессах. Эффективность снижалась при росте числа процессов, а режим с шестнадцатью процессами дополнительно испытывал влияние переподписки. Следовательно, данный алгоритм демонстрирует обмен и случайную маршрутизацию сообщений, но не имеет области эффективного распараллеливания при наличии одного последовательно передаваемого мяча.

8 Список использованных источников

[1] Мараховский В. Б., Розенблюм Л. Я., Яковлев А. В. Моделирование параллельных процессов. Сети Петри. СПб.: Профессиональная литература, 2014.

[2] Message Passing Interface Forum. MPI Standard. Version 4.1. 2023.

ПРИЛОЖЕНИЕ А

Листинг программы «Пляжный волейбол»

```
// Автор: Марк Пархоменко
// Дата: 04.11.2026

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include <mpi.h>

int choose_dest(int rank, int process_count)
{
   int dest;

   do
   {
      dest = rand() % process_count;
   }
   while (dest == rank);

   return dest;
}

int main(int argc, char *argv[])
{
   enum
```

```
{
   TAG_BALL = 1,
   TAG_STOP = 2
};

int rank;
int process_count;
int ball;
int const max_passes = 10;

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &process_count);

srand((unsigned int)time(NULL) + (unsigned int)rank);

if (process_count < 2)
{
   printf("Must have 2 processes\n");
   MPI_Finalize();
   return 0;
}

if (rank == 0)
{
   ball = 1;
   int dest = choose_dest(rank, process_count);
```

```
    MPI_Send(
        &ball,
        1,
        MPI_INT,
        dest,
        TAG_BALL,
        MPI_COMM_WORLD
    );
    printf("rank %d, send ball to rank %d\n",
            rank, dest);

}

bool running = true;

while(running)
{
    int received_msg;
    MPI_Status status;
    MPI_Recv(
        &received_msg,
        1,
        MPI_INT,
        MPI_ANY_SOURCE,
        MPI_ANY_TAG,
        MPI_COMM_WORLD,
        &status
    );
```

```
if (status.MPI_TAG == TAG_BALL)
{
   printf("Sender %d Receiver %d ball %d\n",
         status.MPI_SOURCE, rank, received_msg);

   if (received_msg == max_passes)
   {
      for (int i = 0; i < process_count; i++)
      {
         if (i == rank)
            continue;
         MPI_Send(
            &received_msg,
            1,
            MPI_INT,
            i,
            TAG_STOP,
            MPI_COMM_WORLD
         );
      }
      running = false;

   }
   else
   {
      received_msg++;
      int dest = choose_dest(rank, process_count);
```

```
            MPI_Send(
                &received_msg,
                1,
                MPI_INT,
                dest,
                TAG_BALL,
                MPI_COMM_WORLD
            );
        }

    }
    else if (status.MPI_TAG == TAG_STOP)
    {
        printf("The game was stopped, MPI_TAG %d\n",
                status.MPI_TAG);
        running = false;
    }
  }

  MPI_Finalize();

  return 0;
}
```

ПРИЛОЖЕНИЕ Б

Листинг программы измерения времени

```
// Автор: Марк Пархоменко
// Экспериментальная версия программы Пляжный« волейбол».

#include <errno.h>
#include <limits.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <mpi.h>

enum
{
   TAG_BALL = 1,
   TAG_STOP = 2
};

static volatile int benchmark_sink;

static int choose_dest(int rank, int process_count)
{
   int dest;

   do
   {
```

```
        dest = rand() % process_count;
    }
    while (dest == rank);

    return dest;
}

static bool parse_positive_int(const char *text, int *value)
{
    char *end = NULL;
    errno = 0;
    long parsed = strtol(text, &end, 10);

    if (errno != 0 || end == text || *end != '\0' ||
        parsed <= 0 || parsed > INT_MAX)
    {
        return false;
    }

    *value = (int)parsed;
    return true;
}

static double run_local_round(
    int *message,
    int *scratch,
    int element_count,
    int passes
```

```
)
{
   int *source = message;
   int *destination = scratch;
   size_t message_bytes = (size_t)element_count * sizeof(*message);

   MPI_Barrier(MPI_COMM_WORLD);
   double started_at = MPI_Wtime();

   for (int pass = 1; pass <= passes; pass++)
   {
      source[0] = pass;
      memcpy(destination, source, message_bytes);

      int *temporary = source;
      source = destination;
      destination = temporary;
   }

   MPI_Barrier(MPI_COMM_WORLD);
   double elapsed = MPI_Wtime() - started_at;

   benchmark_sink = source[0] + source[element_count - 1];
   return elapsed;
}

static double run_mpi_round(
   int *message,
```

```
    int element_count,
    int passes,
    int rank,
    int process_count
)
{
    MPI_Barrier(MPI_COMM_WORLD);
    double started_at = MPI_Wtime();

    if (rank == 0)
    {
        message[0] = 1;
        int destination = choose_dest(rank, process_count);
        MPI_Send(
            message,
            element_count,
            MPI_INT,
            destination,
            TAG_BALL,
            MPI_COMM_WORLD
        );
    }

    bool running = true;
    while (running)
    {
        MPI_Status status;
        MPI_Recv(
```

```
    message,
    element_count,
    MPI_INT,
    MPI_ANY_SOURCE,
    MPI_ANY_TAG,
    MPI_COMM_WORLD,
    &status
);

if (status.MPI_TAG == TAG_STOP)
{
    running = false;
}
else if (status.MPI_TAG == TAG_BALL)
{
    if (message[0] >= passes)
    {
        int stop_value = 0;
        for (int destination = 0;
             destination < process_count;
             destination++)
        {
            if (destination != rank)
            {
                MPI_Send(
                    &stop_value,
                    1,
                    MPI_INT,
```

```
                    destination,
                    TAG_STOP,
                    MPI_COMM_WORLD
                );
            }
        }
        running = false;
    }
    else
    {
        message[0]++;
        int destination = choose_dest(rank, process_count);
        MPI_Send(
            message,
            element_count,
            MPI_INT,
            destination,
            TAG_BALL,
            MPI_COMM_WORLD
        );
    }
  }
}

MPI_Barrier(MPI_COMM_WORLD);
double elapsed = MPI_Wtime() − started_at;

benchmark_sink = message[0];
```

```
    return elapsed;
}

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank;
    int process_count;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &process_count);

    if (argc != 5)
    {
        if (rank == 0)
        {
            fprintf(
                stderr,
                "Usage: %s PAYLOAD_INTS PASSES REPETITIONS WARMUPS\n
     ",
                argv[0]
            );
        }
        MPI_Finalize();
        return EXIT_FAILURE;
    }

    int payload_count;
```

```
int passes;
int repetitions;
int warmups;
bool arguments_valid =
    parse_positive_int(argv[1], &payload_count) &&
    parse_positive_int(argv[2], &passes) &&
    parse_positive_int(argv[3], &repetitions) &&
    parse_positive_int(argv[4], &warmups) &&
    payload_count < INT_MAX;

if (!arguments_valid)
{
    if (rank == 0)
    {
        fprintf(stderr, "All arguments must be positive integers.\n");
    }
    MPI_Finalize();
    return EXIT_FAILURE;
}

int element_count = payload_count + 1;
size_t allocation_size = (size_t)element_count * sizeof(int);
int *message = malloc(allocation_size);
int *scratch = process_count == 1 ? malloc(allocation_size) : NULL;

if (message == NULL || (process_count == 1 && scratch == NULL))
{
    fprintf(stderr, "Process %d: unable to allocate message buffer.\n", rank);
```

```
    free(message);
    free(scratch);
    MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    return EXIT_FAILURE;
}

message[0] = 0;
for (int index = 1; index < element_count; index++)
{
    message[index] = index;
}
if (scratch != NULL)
{
    memcpy(scratch, message, allocation_size);
}

srand(1234567u + (unsigned int)rank * 104729u);

int total_rounds = warmups + repetitions;
for (int round = 0; round < total_rounds; round++)
{
    double local_elapsed;
    if (process_count == 1)
    {
        local_elapsed = run_local_round(
            message,
            scratch,
            element_count,
```

```
        passes
    );
}
else
{
    local_elapsed = run_mpi_round(
        message,
        element_count,
        passes,
        rank,
        process_count
    );
}

double maximum_elapsed = 0.0;
MPI_Reduce(
    &local_elapsed,
    &maximum_elapsed,
    1,
    MPI_DOUBLE,
    MPI_MAX,
    0,
    MPI_COMM_WORLD
);

if (rank == 0 && round >= warmups)
{
    int measured_round = round − warmups + 1;
```

```
            const char *mode = process_count == 1 ? "local_copy" : "mpi";
            printf(
                "%d,%zu,%d,%d,%d,%.9f,%s\n",
                payload_count,
                (size_t)payload_count * sizeof(int),
                process_count,
                passes,
                measured_round,
                maximum_elapsed,
                mode
            );
        }
    }

    free(scratch);
    free(message);
    MPI_Finalize();
    return EXIT_SUCCESS;
}
```